MOHAN BABU UNIVERSITY
Sree Sainath Nagar, Tirupati 517 102

COURSE HAND OUT

School:	School of Computing	**Dept.:**	Data Science

Course Code	:	22CS102035					
Course Title	:	**Competitive programming**					
Course Credit Structure		L	T	P	S	C	
		3	-	2	-	4	
Semester		B. Tech IV Semester					
Contact Hours	:	45 Hours					
Instructor	:	Kamal Ahmad					
Instructor's Email	:	kamal.ahmad@faceacademy.in					
Office Hours	:	All days					
Academic Year	:	2025-26					
Date of Issue	:	-					

PROGRAM OUTCOMES

On successful completion of the Program, the graduates of CSE(AIML)Program will be able to:

1. **Engineering knowledge**: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis**: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent

responsibilities relevant to the professional engineering practice.

7. **Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning**: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

CORE

Course Code	Course Title	L	T	P	S	C
22CS102035	**COMPETITIVE PROGRAMMING**	3	-	2	-	4
Pre-Requisite	-					
Anti-Requisite	Design and Analysis of Algorithms					
Co-Requisite	-					

COURSE DESCRIPTION: This course covers algorithm design and analysis, including complexity, mathematical, string, greedy, dynamic programming, recursion, and backtracking techniques. It also explores graph algorithms, branch and bound, and NP-hard/NP-complete problem concepts.

COURSE OUTCOMES: After successful completion of the course, students will be able to:

CO1.	Demonstrate understanding of time and space complexity concepts, and apply mathematical and string algorithms for efficient problem solving.
CO2.	Apply divide and conquer and greedy strategies to design and analyze efficient algorithms for sorting, searching, and optimization problems.
CO3.	Develop dynamic programming and bitwise algorithm solutions for complex computational problems involving overlapping subproblems and bit manipulation.
CO4.	Utilize recursion and backtracking techniques to explore solution spaces and solve constraint satisfaction problems effectively.
CO5.	Analyze graph algorithms including shortest path, cycle detection, and connectivity, and apply advanced concepts such as branch and bound and NP-completeness theory.

CO6.	Work independently to solve problems with effective communication.

CO-PO-PSO Mapping Table:

Learning Outcomes	Program Outcomes												Program Specific Outcomes			
	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO 10	PO 11	PO 12	PSO1	PSO2	PSO3	PSO4
CO1	3	2	-	-	3	-	-	-	-	-	-	-	-	-	3	-
CO2	3	2	-	-	3	-	-	-	-	-	-	-	-	-	3	-
CO3	3	3	3	3	3	-	-	-	-	-	-	-	-	-	3	-
CO4	3	2	2	2	3	-	-	-	-	-	-	-	-	-	3	-
CO5	3	3	3	3	3	-	-	-	-	-	-	-	-	-	3	-
CO6	-	-	-	-	-	-	-	3	3	-	-	-	-	-	-	-
Course Correlation Mapping	3	3	3	3	3	-	-	3	3	-	-	-	-	-	3	-

Correlation Levels: 3: High; 2: Medium; 1: Low

COURSE CONTENT

Module 1:	**TIME AND SPACE COMPLEXITY, MATHEMATICAL AND STRING ALGORITHMS**	***(9 Periods)***
Time and space complexity: Big-O, Big-Ω, Big-Θ notations, best, average, worst-case analysis, Asymptotic analysis, growth functions, complexity classes, time estimation, **Mathematical algorithms:** GCD, LCM, modular arithmetic, prime checks, Sieve of Eratosthenes, binary exponentiation, modular power, modular inverse, base conversion, digit operations. **String algorithms:** naive pattern matching, KMP, Rabin-Karp, Z-algorithm, longest palindromic substring, Introduction of Debugging tools such as GDB and online IDEs and code visualizer		
Module 2:	**DIVIDE AND CONQUER, GREEDY ALGORITHMS**	***(9 Periods)***
Divide and conquer strategy, recurrence relations, Master Theorem, Algorithms: merge sort, quick sort, binary search, closest pair of points, matrix multiplication; Greedy strategy, activity selection, fractional knapsack, Huffman coding, MST (Prim's, Kruskal's), job sequencing, greedy-choice property, comparison with dynamic programming.		
Module 3:	**DYNAMIC PROGRAMMING AND BITWISE ALGORITHMS**	***(8 Periods)***
Dynamic programming concepts, overlapping subproblems, memoization, principle of optimality, Classical problems: 0/1 knapsack, LCS, matrix chain multiplication, coin change, LIS, Tabulation vs memoization, bit manipulation: AND, OR, XOR, set/clear/check bits, power of two, bitwise problems like unique elements, subset generation, intro to bitmask DP.		

Module 4:	**RECURSION AND BACKTRACKING**	***(8 Periods)***
Recursion basics, recursive tree, tail recursion, factorial, Fibonacci, Towers of Hanoi, Backtracking concepts, solution space exploration, pruning techniques, Backtracking problems: N-Queens, Sudoku solver, Rat in a Maze, subset sum, word search, comparison with pure recursion.		
Module 5:	**GRAPH ALGORITHMS BRANCH AND BOUND, NP-HARD AND NP-COMPLETE**	***(11 Periods)***
Shortest path algorithms: introduction to Dijkstra's, Bellman-Ford (conceptual only); Cycle detection methods, checking connectivity in graphs, Topological sorting, Advanced concepts: Trie (intro), suffix arrays (conceptual). **Branch and Bound:** Bounding functions, solving TSP, 0/1 knapsack; NP-Hard and NP-Complete problems: SAT, Clique, subset sum, reductions, P vs NP.		
		Total Periods: 45

EXPERIENTIAL LEARNING

LIST OF EXERCISES: **(All source code should be maintained and version-controlled on GitHub)**

- **TIME AND SPACE COMPLEXITY, MATHEMATICAL AND STRING ALGORITHMS**

  Leetcode: #1250. Check If It Is a Good Array

  Given an array nums of positive integers. Your task is to select some subset of nums, multiply each element by an integer and add all these numbers. The array is said to be **good** if you can obtain a sum of 1 from the array by any possible subset and multiplicand.

  Return True if the array is **good** otherwise return False.

  Example 1:

  Input: nums = [12,5,7,23]

  Output: true

  Explanation: Pick numbers 5 and 7.

  5*3 + 7*(-2) = 1

  Example 2:

  Input: nums = [29,6,10]

  Output: true

  Explanation: Pick numbers 29, 6 and 10.

  29*1 + 6*(-3) + 10*(-1) = 1

  Example 3:

  Input: nums = [3,6]

  Output: false

  Constraints:

  - 1 <= nums.length <= 10^5
  - 1 <= nums[i] <= 10^9

  ✓ This problem reinforces the concept of Linear Diophantine Equations and their connection with the Greatest Common Divisor (GCD). It highlights that an array is good if the GCD of all elements is 1, making it a valuable lesson in number theory.

- **TIME AND SPACE COMPLEXITY, MATHEMATICAL AND STRING ALGORITHMS**

  LeetCode: **#2344. Minimum Deletions to Make Array Divisible**

  You are given two positive integer arrays nums and numsDivide. You can delete any number of elements from nums.

  Return *the **minimum** number of deletions such that the **smallest** element in* nums ***divides*** *all the elements of* numsDivide. If this is not possible, return -1.

  Note that an integer x divides y if y % x == 0.

  Example 1:

  Input: nums = [2,3,2,4,3], numsDivide = [9,6,9,3,15]

  Output: 2

  Explanation:

  The smallest element in [2,3,2,4,3] is 2, which does not divide all the elements of numsDivide.

  We use 2 deletions to delete the elements in nums that are equal to 2 which makes nums = [3,4,3].

  The smallest element in [3,4,3] is 3, which divides all the elements of numsDivide.

  It can be shown that 2 is the minimum number of deletions needed.

  Example 2:

  Input: nums = [4,3,6], numsDivide = [8,2,6,10]

  Output: -1

  Explanation:

  We want the smallest element in nums to divide all the elements of numsDivide.

  There is no way to delete elements from nums to allow this.

  Constraints:

  - 1 <= nums.length, numsDivide.length <= 10^5
  - 1 <= nums[i], numsDivide[i] <= 10^9

  ✓ This problem enhances understanding of stack-based validation and greedy algorithms for maintaining syntactic correctness. It helps learners manage open-close matching logic and handle unbalanced expressions efficiently.

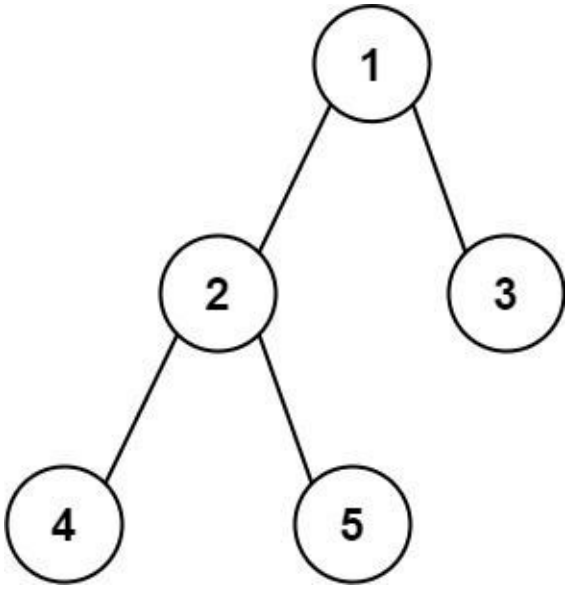
- **DIVIDE AND CONQUER, GREEDY ALGORITHMS**

Leetcode: #543. Diameter of Binary Tree
Given the root of a binary tree, return *the length of the **diameter** of the tree*.

The **diameter** of a binary tree is the **length** of the longest path between any two nodes in a tree. This path may or may not pass through the root.

The **length** of a path between two nodes is represented by the number of edges between them.

Example 1:



Input: root = [1,2,3,4,5]

Output: 3

Explanation: 3 is the length of the path [4,2,1,3] or [5,2,1,3].

Example 2:

Input: root = [1,2]

Output: 1

Constraints:

- The number of nodes in the tree is in the range [1, 10^4].
- -100 <= Node.val <= 100

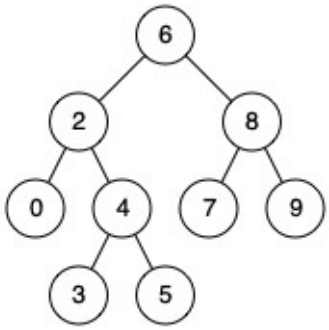
✓ This problem helps learners understand tree traversal, particularly depth-first search (DFS). It develops logic to compute height of subtrees and track maximum path length between any two nodes. Ideal for mastering recursive strategies and handling tree-based interview problems.

- **DIVIDE AND CONQUER, GREEDY ALGORITHMS**

  Leetcode: #235. Lowest Common Ancestor of a Binary Search Tree
  Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.

  According to the definition of LCA on Wikipedia: "The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow **a node to be a descendant of itself**)."

  Example 1:

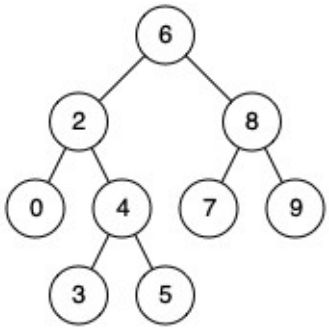


  Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8

  Output: 6

  Explanation: The LCA of nodes 2 and 8 is 6.

  Example 2:



  Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4

  Output: 2

  Explanation: The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself according to the LCA definition.

  Example 3:

  Input: root = [2,1], p = 2, q = 1

  Output: 2

  Constraints:

  - The number of nodes in the tree is in the range [2, 10^5].
  - -10^9 <= Node.val <= 10^9

- **DYNAMIC PROGRAMMING AND BITWISE ALGORITHMS**

  LeetCode: **#210. Course Schedule II**

  There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [a_i, b_i] indicates that you **must** take course b_i first if you want to take course a_i.

  - For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

  Return *the ordering of courses you should take to finish all courses*. If there are many valid answers, return **any** of them. If it is impossible to finish all courses, return **an empty array**.

  Example 1:

  Input: numCourses = 2, prerequisites = [[1,0]]

  Output: [0,1]

  Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So the correct course order is [0,1].

  Example 2:

  Input: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]

  Output: [0,2,1,3]

  Explanation: There are a total of 4 courses to take. To take course 3 you should have finished both courses 1 and 2. Both courses 1 and 2 should be taken after you finished course 0.

  So one correct course order is [0,1,2,3]. Another correct ordering is [0,2,1,3].

  Example 3:

  Input: numCourses = 1, prerequisites = []

  Output: [0]

  Constraints:

  - 1 <= numCourses <= 2000
  - 0 <= prerequisites.length <= numCourses * (numCourses - 1)
  - prerequisites[i].length == 2
  - 0 <= a_i, b_i < numCourses
  - a_i != b_i
  - All the pairs [a_i, b_i] are **distinct**.

  ✓ This problem strengthens your understanding of topological sorting, especially using BFS (Kahn's algorithm) or DFS. It helps learners grasp cycle detection in directed graphs, critical for scheduling and dependency resolution problems.

- **DYNAMIC PROGRAMMING AND BITWISE ALGORITHMS**

Leetcode: #200. Number of Islands

Given an m x n 2D binary grid grid which represents a map of '1's (land) and '0's (water), return *the number of islands*.
An **island** is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.
Example 1:
Input: grid = [
["1","1","1","1","0"],
["1","1","0","1","0"],
["1","1","0","0","0"],
["0","0","0","0","0"]
]
Output: 1
Example 2:
Input: grid = [
["1","1","0","0","0"],
["1","1","0","0","0"],
["0","0","1","0","0"],
["0","0","0","1","1"]
]
Output: 3

Constraints:
- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 300
- grid[i][j] is '0' or '1'.

✓ This problem enhances understanding of graph traversal techniques like DFS or BFS in 2D matrices. It builds foundational skills for flood fill algorithms, useful in image processing and geographic data analysis.

- **RECURSION AND BACKTRACKING**

Leetcode: #35. Search Insert Position

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order. You must write an algorithm with O(log n) runtime complexity.

Example 1:
Input: nums = [1,3,5,6], target = 5
Output: 2
Example 2:
Input: nums = [1,3,5,6], target = 2
Output: 1
Example 3:
Input: nums = [1,3,5,6], target = 7
Output: 4

Constraints:

- 1 <= nums.length <= 10^4
- -10^4 <= nums[i] <= 10^4
- nums contains **distinct** values sorted in **ascending** order.
- -10^4 <= target <= 10^4

✓ This problem strengthens understanding of binary search, a fundamental divide-and-conquer technique. It trains learners to efficiently handle insert position logic within sorted data structures. The concept is widely applicable in search engines, database indexing, and algorithm optimization tasks.

- **RECURSION AND BACKTRACKING**

Leetcode: #324. Wiggle Sort II

Given an integer array nums, reorder it such that nums[0] < nums[1] > nums[2] < nums[3]....
You may assume the input array always has a valid answer.

Example 1:
Input: nums = [1,5,1,1,6,4]
Output: [1,6,1,5,1,4]
Explanation: [1,4,1,5,1,6] is also accepted.
Example 2:
Input: nums = [1,3,2,2,3,1]
Output: [2,3,1,3,1,2]

Constraints:

- 1 <= nums.length <= $5 * 10^4$
- 0 <= nums[i] <= 5000
- It is guaranteed that there will be an answer for the given input nums.

✓ This problem develops an understanding of array rearrangement patterns, particularly wiggle sort. It enhances knowledge of median finding, index mapping, and three-way partitioning techniques. Useful in scenarios where alternating highs and lows are required, such as stock price modeling or load balancing.

- **GRAPH ALGORITHMS BRANCH AND BOUND, NP-HARD AND NP-COMPLETE**

  LeetCode: **#210. Course Schedule II**

  There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [a_i, b_i] indicates that you **must** take course b_i first if you want to take course a_i.

  - For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

  Return *the ordering of courses you should take to finish all courses*. If there are many valid answers, return **any** of them. If it is impossible to finish all courses, return **an empty array**.

  Example 1:

  Input: numCourses = 2, prerequisites = [[1,0]]

  Output: [0,1]

  Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So the correct course order is [0,1].

  Example 2:

  Input: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]

  Output: [0,2,1,3]

  Explanation: There are a total of 4 courses to take. To take course 3 you should have finished both courses 1 and 2. Both courses 1 and 2 should be taken after you finished course 0.

  So one correct course order is [0,1,2,3]. Another correct ordering is [0,2,1,3].

  Example 3:

  Input: numCourses = 1, prerequisites = []

  Output: [0]

  Constraints:

  - 1 <= numCourses <= 2000
  - 0 <= prerequisites.length <= numCourses * (numCourses - 1)
  - prerequisites[i].length == 2
  - 0 <= a_i, b_i < numCourses
  - a_i != b_i
  - All the pairs [a_i, b_i] are **distinct**.

  ✓ This problem strengthens your understanding of topological sorting, especially using BFS (Kahn's algorithm) or DFS. It helps learners grasp cycle detection in directed graphs, critical for scheduling and dependency resolution problems.

- **GRAPH ALGORITHMS BRANCH AND BOUND, NP-HARD AND NP-COMPLETE**

  Leetcode: #200. Number of Islands

  Given an m x n 2D binary grid grid which represents a map of '1's (land) and '0's (water), return *the number of islands*.
  An **island** is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.
  Example 1:
  Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
  ]
  Output: 1
  Example 2:
  Input: grid = [
  ["1","1","0","0","0"],
  ["1","1","0","0","0"],
  ["0","0","1","0","0"],
  ["0","0","0","1","1"]
  ]
  Output: 3

  Constraints:
  - m == grid.length
  - n == grid[i].length
  - 1 <= m, n <= 300
  - grid[i][j] is '0' or '1'.

  ✓ This problem enhances understanding of graph traversal techniques like DFS or BFS in 2D matrices. It builds foundational skills for flood fill algorithms, useful in image processing and geographic data analysis.

RESOURCES

TEXT BOOKS:

- Ellis Horowitz, Sartaj Sahni, and Sanguthevar Rajasekaran, Fundamentals of Computer Algorithms, 2nd Edition, Universities Press, 2008.
- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein, Introduction to Algorithms,3rd Edition, MIT Press, 2009.

REFERENCE BOOKS:

1. Michael T. Goodrich and Roberto Tamassia, Algorithm Design and Applications, Wiley, 2014.

2. Alfred V. Aho, John E. Hopcroft and Jeffrey D. Ullman, The Design and Analysis of Computer Algorithms, Pearson, 2006.

SOFTWARE/TOOLS:
1. Software: JDK 1.8
2. Operating System: Windows/ Linux
3. VS Code / Online IDE / Eclipse

VIDEO LECTURES:

- https://www.youtube.com/watch?v=0IAPZzGSbME&list=PLDN4rrl48XKpZkf03iYFl-O29szjTrs_O
- https://www.youtube.com/watch?v=u8JZ9gU5o4g&list=PLxCzCOWd7aiHcmS4i14bI0VrMbZTUvlTa
- https://www.youtube.com/watch?v=B6L7hh_H1wo&list=PL1QH9gyQXfgs7foRxIbIH8wmJyDh5QzAm
- https://www.youtube.com/watch?v=u8JZ9gU5o4g&list=PLxCzCOWd7aiHcmS4i14bI0VrMbZTUvlTa
- https://www.youtube.com/watch?v=UgIQN5S9-Ic&list=PLBlnK6fEyqRhoF3cPp0mgOZPuXeu84nAd
- https://www.youtube.com/watch?v=WOCSibinb18&list=PLmAmHQ-_5ySx5Pov1ZJmHebsVuiBvUflf

WEB RESOURCES:

- https://www.geeksforgeeks.org/data-structures/
- https://www.programiz.com/dsa
- https://www.tutorialspoint.com/data_structures_algorithms/index.htm
- https://visualgo.net/en (for visualizing trees, graphs, heaps, etc.)
- https://www.interviewbit.com/data-structure-interview-questions/

Course Delivery Schedule:

S. No	Topic	Contact Hours	CO Mapping	Pedagogy	Resources
TIME AND SPACE COMPLEXITY, MATHEMATICAL AND STRING ALGORITHMS					
1	Time Complexity	1	CO1	PPT and Live Coding	TB 1
2	Time Complexity	1	CO1	Interactive discussion with code demos	RB 2
3	Growth of Functions	1	CO1	Interactive discussion with code demos	TB 2
4	Math Algorithms	1	CO1	PPT and Live Coding	RB 1
5	Problems on Math Algorithms - 1	1	CO1	Interactive discussion with code demos	TB 2
6	Problems on Math Algorithms - 2	1	CO1	Interactive discussion with code demos	TB 1
7	Problems on Math Algorithms - 3	1	CO1	Interactive discussion with code demos	RB 2
8	Problems on Math Algorithms - 4	1	CO1	Visual simulation + code practice	TB 1
9	Problems on Math Algorithms - 5	1	CO1	Visual simulation + code practice	TB 1
10	Problems on Math Algorithms - 6	1	CO1	Live walkthrough + dry run	RB 2
11	Problems on Math Algorithms - 7	1	CO1	Live walkthrough + dry run	TB 2
12	String Algorithms	1	CO1	PPT and Live Coding	RB 1
13	Problems on String Algorithms - 1	1	CO1	Interactive discussion with code demos	RB 2

14	Problems on String Algorithms - 2	1	CO1	Interactive discussion with code demos	TB 2
15	Problems on String Algorithms - 3	1	CO1	Visual simulation + code practice	RB 2
16	Problems on String Algorithms - 4	1	CO1	Visual simulation + code practice	RB 1
17	Problems on String Algorithms - 5	1	CO1	Live walkthrough + dry run	TB 2
18	Problems on String Algorithms - 6	1	CO1	Live walkthrough + dry run	RB 2
DIVIDE AND CONQUER, GREEDY ALGORITHMS					
19	Divide and Conquer	1	CO2	PPT and Live Coding	RB 1
20	Problems on Divide and Conquer - 1	1	CO2	Interactive discussion with code demos	RB 1
21	Problems on Divide and Conquer - 2	1	CO2	Interactive discussion with code demos	RB 2
22	Problems on Divide and Conquer - 3	1	CO2	Interactive discussion with code demos	TB 1
23	Problems on Divide and Conquer - 4	1	CO2	Visual simulation + code practice	RB 2
24	Problems on Divide and Conquer - 5	1	CO2	Visual simulation + code practice	TB 2
25	Problems on Divide and Conquer - 6	1	CO2	Live walkthrough + dry run	RB 1
26	Greedy Algorithms	1	CO2	PPT and Live Coding	RB 1
27	Problems on Greedy Algorithms - 1	1	CO2	Visual simulation + code practice	TB 1
28	Problems on Greedy Algorithms - 2	1	CO2	Visual simulation + code practice	RB 2
29	Problems on Greedy Algorithms - 3	1	CO2	Visual simulation + code practice	TB 1

30	Problems on Greedy Algorithms - 4	1	CO2	Live walkthrough + dry run	RB 2
31	Problems on Greedy Algorithms - 5	1	CO2	Live walkthrough + dry run	TB 2
32	Problems on Greedy Algorithms - 6	1	CO2	Live walkthrough + dry run	RB 1
33	Problems on Greedy Algorithms - 7	1	CO2	Live walkthrough + dry run	TB 2
34	Problems on Greedy Algorithms - 8	1	CO2	Interactive discussion with code demos	RB 1
35	Problems on Greedy Algorithms - 9	1	CO2	Interactive discussion with code demos	TB 1
DYNAMIC PROGRAMMING AND BITWISE ALGORITHMS					
36	Dynamic Programming	1	CO3	PPT and Live Coding	RB 2
37	Problems on Dynamic Programming - 1	1	CO3	Interactive discussion with code demos	RB 1
38	Problems on Dynamic Programming - 2	1	CO3	Interactive discussion with code demos	RB 1
39	Problems on Dynamic Programming - 3	1	CO3	Interactive discussion with code demos	RB 2
40	Problems on Dynamic Programming - 4	1	CO3	Interactive discussion with code demos	TB 2
41	Problems on Dynamic Programming - 5	1	CO3	Visual simulation + code practice	RB 1
42	Problems on Dynamic Programming - 6	1	CO3	Visual simulation + code practice	TB 1
43	Problems on Dynamic Programming - 7	1	CO3	Live walkthrough + dry run	RB 2
44	Problems on Dynamic Programming - 8	1	CO3	Live walkthrough + dry run	RB 1

45	Problems on Dynamic Programming - 9	1	CO3	Live walkthrough + dry run	RB 2
46	Problems on Dynamic Programming - 10	1	CO3	Visual simulation + code practice	RB 1
47	Problems on Dynamic Programming - 11	1	CO3	Visual simulation + code practice	TB 2
48	Bitwise Algorithms	1	CO3	PPT and Live Coding	RB 1
49	Problems on Bitwise Algorithms - 1	1	CO3	Interactive discussion with code demos	TB 1
50	Problems on Bitwise Algorithms - 2	1	CO3	Interactive discussion with code demos	RB 1
51	Problems on Bitwise Algorithms - 3	1	CO3	Visual simulation + code practice	TB 2
52	Problems on Bitwise Algorithms - 4	1	CO3	Visual simulation + code practice	RB 2
53	Problems on Bitwise Algorithms - 5	1	CO3	Live walkthrough + dry run	RB 1
	RECURSION AND BACKTRACKING				
54	Recursion and Backtracking	1	CO4	PPT and Live Coding	TB 2
55	Problems on Recursion and Backtracking - 1	1	CO4	Interactive discussion with code demos	TB 1
56	Problems on Recursion and Backtracking - 2	1	CO4	Interactive discussion with code demos	RB 1
57	Problems on Recursion and Backtracking - 3	1	CO4	Interactive discussion with code demos	RB 2
58	Problems on Recursion and Backtracking - 4	1	CO4	Interactive discussion with code demos	RB 2
59	Problems on Recursion and Backtracking - 5	1	CO4	Visual simulation + code practice	RB 1

60	Problems on Recursion and Backtracking - 6	1	CO4	Visual simulation + code practice	TB 2
61	Problems on Recursion and Backtracking - 7	1	CO4	Live walkthrough + dry run	RB 2
62	Problems on Recursion and Backtracking - 8	1	CO4	Live walkthrough + dry run	TB 1
63	Problems on Recursion and Backtracking - 9	1	CO4	Live walkthrough + dry run	RB 2
64	Problems on Recursion and Backtracking - 10	1	CO4	Live walkthrough + dry run	TB 2
65	Problems on Recursion and Backtracking - 11	1	CO4	Live walkthrough + dry run	RB 1
GRAPH ALGORITHMS BRANCH AND BOUND, NP-HARD AND NP-COMPLETE					
66	Graphs	1	CO5	PPT and Live Coding	RB 1
67	Graphs	1	CO5	PPT and Live Coding	TB 2
68	Graphs - Shortest Path Algorithms	1	CO5	Interactive discussion with code demos	TB 1
69	Graphs - Shortest Path Algorithms	1	CO5	Interactive discussion with code demos	TB 2
70	Suffix arrays	1	CO5	Interactive discussion with code demos	RB 1
71	Branch and Bound	1	CO5	PPT and Live Coding	RB 2
72	Branch and Bound	1	CO5	PPT and Live Coding	RB 1
73	NP-HARD AND NP-COMPLETE	1	CO5	PPT and Live Coding	TB 2
74	NP-HARD AND NP-COMPLETE	1	CO5	Interactive discussion with code demos	TB 1

75	NP-HARD AND NP-COMPLETE	1	CO5	Interactive discussion with code demos	RB 2

COURSE EVALUATION:

Type of Exam	Contents	No. of questions	Time duration	Marks	Platform	Question type
CIAT-I	Module I	20+3	120 minutes	50	Examly	MCQ + Coding
CIAT-II	Modules II and III	20+3	120 minutes	50	Examly	MCQ + Coding
CIAT-III	Modules IV and V	20+3	120 minutes	50	Examly	MCQ + Coding
End Semester Exam	All Modules	30+5	180 minutes	100 (Scale to 50)	Examly	MCQ + Coding
	Total Marks			**100 (Average of CIAT -I,II,III – 50 Marks + End Semester – 50 Marks)**		

MODEL QUESTION PAPER

CODE No.: 22CS102035

III B.Tech. I Semester (MBU-22) Regular Examinations
November – 2022

COMPETITIVE PROGRAMMING

Part A

Multiple choice questions
30 X 1 = 30

S.No	Question	BL	CO
1	The time complexity of the binary search algorithm in the worst case is: a) O(n) b) O(log n) c) O(n²) d) O(1)	L1	CO1
2	Which of the following is an example of a mathematical algorithm? a) Fibonacci b) Sieve of Eratosthenes c) Activity Selection d) Dijkstra	L1	CO1
3	The GCD of 36 and 60 using the Euclidean algorithm is: a) 6 b) 12 c) 18 d) 24	L1	CO1
4	The time complexity of the naive pattern matching algorithm is: a) O(n + m) b) O(n*m) c) O(n log n) d) O(1)	L2	CO1
5	Which algorithm is used for finding all prime numbers up to n? a) Sieve of Eratosthenes b) KMP c) Rabin-Karp d) Dijkstra	L2	CO1
6	In the KMP string matching algorithm, the LPS array is used to: a) Store pattern characters b) Skip unnecessary comparisons c) Count matches d) Generate substrings	L2	CO1
7	In the fractional knapsack problem, the greedy approach works because: a) The items are divisible b) The items are indivisible c) The problem is NP-complete d) The weights are equal	L2	CO2

8	The divide and conquer strategy works by: a) Dividing the problem into subproblems and combining results b) Selecting the best local option at each step c) Storing previous results for reuse d) Recursively reducing the problem to one element	L2	CO2
9	For Binary search of a sorted array of size n, the best case time complexity is: a) O(n) b) O(log n) c) O(1) d) O(n log n)	L1	CO2
10	What is the space complexity of merge sort? a) O(1) b) O(n) c) O(log n) d) $O(n^2)$	L1	CO2
11	Which of the following algorithms is based on the divide and conquer strategy? a) Merge Sort b) Bubble Sort c) Linear Search d) Insertion Sort	L1	CO2
12	Which algorithm is used to find the minimum spanning tree? a) BFS b) Kruskal c) DFS d) Bellman-Ford	L1	CO2
13	In dynamic programming, memoization is used to: a) Store intermediate results to avoid recomputation b) Divide the problem into subproblems c) Make greedy choices d) Prune the solution space	L3	CO3
14	Which of the following operations is bitwise XOR? a) & b) | c) ^ d) ~	L1	CO3
15	Which of the following is true about dynamic programming? a) No overlapping subproblems b) Works only for greedy problems c) Stores solutions to subproblems d) Always exponential time	L1	CO3
16	In 0/1 Knapsack problem, if we solve subproblems first and store their solutions, this approach is called: a) Recursion b) Memoization c) Brute force d) Greedy	L3	CO3
17	The Longest Common Subsequence (LCS) problem is solved using: a) Divide and Conquer b) Dynamic Programming c) Greedy Algorithm d) Backtracking	L1	CO3

18	Which of the following bitwise operations is used to toggle a bit? a) AND b) OR c) XOR d) Left Shift	L2	CO3
19	How many solutions does the N-Queens problem have for N=4? a) 1 b) 2 c) 4 d) 8	L1	CO4
20	What does a recursive tree represent? a) Tree data structure implementation b) Memory allocation of heap c) All recursive calls made by a function d) Compiler optimization steps	L1	CO4
21	In a backtracking approach, what is pruning? a) Ignoring all possibilities b) Exploring all solutions c) Eliminating paths that cannot lead to a solution d) Sorting data	L2	CO4
22	Tail recursion is a recursion in which: a) Function calls itself first b) Recursive call is the last operation c) Function never calls itself d) Multiple recursive calls exist	L1	CO4
23	Which of the following is a disadvantage of simple recursive fibonacci a) Uses no stack memory b) Time complexity is exponential c) Cannot be optimized d) Always faster than iteration	L2	CO4
24	What is the base case in recursion: a) first recursive call b) condition to stop recursion c) largest input case d) recursive formula	L2	CO4
25	Dijkstra's algorithm cannot handle: a) Positive edge weights b) Negative edge weights c) Connected graphs d) Weighted graphs	L1	CO5
26	Which of the following is an NP-Complete problem? a)Sorting b) Subset Sum c) Fibonacci d) BFS traversal	L2	CO5
27	Which algorithm is used to find the shortest path in a graph with non-negative weights? a) Prim's b) Bellman-Ford c) Dijkstra d) Kruskal	L1	CO5
28	Which of the following algorithms is based on branch and bound technique? a) Merge Sort b) 0/1 Knapsack c) Binary Search d) Floyd-Warshall	L1	CO5

29	Which of the following checks if a graph has cycles? a) BFS b) DFS c) Both BFS and DFS d) Dijkstra	L2	CO5
30	Branch and bound is mainly used to solve: a) searching problems only b) Graph traversal problems c) optimization problems d) Sorting problems	L1	CO5

Part B

Programming questions
5 X 14 =70

S.No	Question	BL

1	**Festival Arrangement** A city is planning a festival and wants to display special numbers on banners. A number is considered *special* if it is greater than 1 and has no divisors other than 1 and itself. Given a number N, list all such special numbers less than or equal to N. The solution should be efficient enough to handle large values of N. Input: A single integer N Output: Space-separated special numbers less than or equal to N Constraints: $1 \leq N \leq 10^7$ Example: Input: 15 Output: 2 3 5 7 11 13	**L2**

2	**Cargo Loading System** A logistics company needs to load cargo onto a truck with limited capacity. Each cargo item has a weight and a profit value. Your task is to choose a subset of items such that the total profit is maximized without exceeding the truck's weight capacity. Each item can be selected at most once. Input: An integer N and capacity W, followed by N lines each containing the weight and profit of an item Output: Maximum achievable profit Constraints: $1 \leq N \leq 10^3$ $1 \leq W \leq 10^5$ Example: Input: 4 7 1 1 3 4 4 5 5 7 Output: 9	**L3**

3	**Delivery Time Estimator** A delivery service operates from a central hub and needs to estimate the minimum time required to deliver packages to all destinations. The travel time between locations is non-negative. Given the map of delivery routes and a starting hub, determine the minimum time required to reach each destination. Input: Integers N, M, and the source hub, followed by M routes represented as (start, end, travel time) Output: Minimum time to reach each destination from the source hub Constraints: $1 \leq N, M \leq 10^5$ Example: Input: 5 6 1 1 2 2 1 3 5 2 3 1 2 4 6 3 5 2 4 5 1 Output: 0 2 3 8 5	**L3**

4	**N-Queens Problem** You are given an integer N. Your task is to place N queens on an N × N chessboard such that no two queens attack each other. A queen can attack another queen if they are placed: • In the same row • In the same column • On the same diagonal **Input Format** • A single integer N **Output Format** • Print all valid board configurations • Each configuration should be represented as N strings of length N • Use 'Q' to denote a queen and '.' to denote an empty cell • Separate different solutions with a blank line **Constraints** • $1 \leq N \leq 10$ **Example** Input 4 Output . Q . . . . . Q Q . . . . . Q .	**L4**

5	**Network Reliability Check** A company's internal network consists of multiple machines connected by cables. Due to security concerns, the company wants to check whether the network contains any circular connections that could cause data loops. Given the network configuration, determine whether such a loop exists. Input: Two integers N and M, followed by M connections between machines Output: Print YES if a loop exists, otherwise print NO Constraints: $1 \leq N, M \leq 10^5$ Example: Input: 4 4 1 2 2 3 3 4 4 2 Output: YES	**L4**

Target for Course Outcome Attainment:

Course Outcomes	Attainment Target (%)
CO1	60%
CO2	60%
CO3	60%
CO4	60%
CO5	60%

Note:

Any additional information further to this hand out will be announced in the class.

Signature of the Course Instructor	Signature of the Chairperson BOS